

WEB APPLICATION SECURITY ASSESSMENT

Local Vulnerability Assessment and Remediation Lab

Student Cybersecurity Project Report

Target: Custom Flask Web Application

Target Address: 127.0.0.1

Target Port: 5000

Environment: Kali Linux — Authorized Local Laboratory

Testing Tools: Burp Suite, Firefox, Flask, SQLite

This assessment was performed against a deliberately vulnerable application created for controlled security testing. The work focuses on demonstrating vulnerabilities, collecting evidence, applying practical remediation, and retesting the application to verify the fixes.

1. Introduction

Web applications commonly process authentication credentials, user-controlled input, and access-controlled resources. Weaknesses in these areas can allow an attacker to bypass authentication logic, execute unintended client-side code, or access information belonging to another user. This project presents a controlled security assessment of a locally hosted Flask application intentionally designed with common web security weaknesses.

The assessment was conducted entirely within a local Kali Linux laboratory. Burp Suite was used to inspect and modify HTTP requests, while the application itself used Flask and SQLite to provide the vulnerable functionality. After each vulnerability was demonstrated, the relevant application logic was corrected and the behavior was tested again.

2. Objective

The primary objective of the project was to demonstrate an end-to-end vulnerability assessment rather than only performing exploitation. The assessment therefore covered reconnaissance of the local application, manual testing of authentication and user-controlled input, identification of broken access control, collection of HTTP evidence, remediation of the vulnerable code, and retesting of the corrected application.

A further objective was to understand the difference between proving a vulnerability and proving that a vulnerability has been fixed. For this reason, the final evidence includes both vulnerable behavior and post-remediation behavior where applicable.

3. Scope

The scope was limited to the intentionally vulnerable Flask application running on the local machine. No external hosts or third-party systems were targeted.

Item	Details
Target	Custom Flask security assessment lab
Target Address	127.0.0.1
Target Port	5000
Database	SQLite
Environment	Kali Linux local laboratory
Authorization	Intentionally vulnerable application owned/controlled for testing

4. Lab Environment

The application was executed locally using Python and Flask. SQLite was used as the backend database, containing test accounts such as admin, pritam, and user2. The application exposed a login endpoint, a search endpoint, and a profile endpoint. Burp Suite was positioned between the browser and the local application so that requests could be inspected and replayed.

Component	Purpose
Kali Linux	Security testing environment
Python / Flask	Local web application runtime
SQLite	Application database
Burp Suite	HTTP interception, request modification and validation
Firefox	Browser-based application interaction

5. Tools Used

Burp Suite was the main testing tool because the assessment depended on observing how the application responded to modified HTTP requests. Firefox was used to generate normal application traffic and to validate browser-side behavior. The Flask development server provided the local target, while SQLite stored the test user records.

6. Methodology

The assessment followed a vulnerability-assessment workflow beginning with application setup and reconnaissance. Normal application requests were first observed so that the structure of the login, search, and profile functions could be understood. The requests were then modified in a controlled manner to test SQL injection, cross-site scripting, and broken access control. Evidence was captured from the resulting HTTP responses and browser behavior. The vulnerable code was subsequently corrected, and the same functionality was retested to determine whether the observed weakness had been removed.

7. Reconnaissance and Application Mapping

The first stage was to confirm that the local application was reachable and identify the exposed web service. Because the target was intentionally hosted on the local machine, reconnaissance was limited to the local address and port used by the Flask server.

Application startup command:

```
python3 app.py
```

Once the Flask server was running, the application was accessed through the browser at `http://127.0.0.1:5000`. Burp Suite was then used to observe the HTTP traffic generated by the application. The available functions identified during mapping were the login endpoint, the search endpoint, and the profile endpoint.

Local target:

```
http://127.0.0.1:5000
```

The application was then tested manually. The login function was assessed for SQL injection, the search function was assessed for reflected client-side script execution, and the profile function was assessed for object-level authorization. The same functions were later used for remediation verification.

8. Vulnerability Findings

Three security weaknesses were identified in the intentionally vulnerable application. The findings were not treated as isolated payload demonstrations; each was documented in terms of observed behavior, security impact, remediation, and retesting. This approach provides a more complete assessment because it shows both the weakness and the resulting security improvement.

8.1. SQL Injection

The login endpoint was found to be vulnerable to SQL injection because user-controlled values were incorporated into the authentication query without safe parameter handling in the vulnerable version of the application. During testing, a modified username value was supplied through Burp Suite while an intentionally incorrect password was retained. The application returned a successful authentication response and identified the authenticated account as admin.

Test evidence:

username=admin' OR '1'='1' -- &password=wrongpassword

The screenshot displays the Burp Suite interface with the 'Request' and 'Response' tabs selected. The 'Request' tab shows a POST request to /login with various headers and a modified username payload. The 'Response' tab shows the server's response, which is a 200 OK status with HTML content indicating a successful login for the admin user.

Request	Response
1 POST /login HTTP/1.1	1 HTTP/1.1 200 OK
2 Host: 127.0.0.1:5000	2 Server: Werkzeug/3.1.8 Python/3.13.14
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0	3 Date: Thu, 20 Aug 2026 10:45:35 GMT
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8	4 Content-Type: text/html; charset=utf-8
5 Accept-Language: en-US,en;q=0.5	5 Content-Length: 45
6 Accept-Encoding: gzip, deflate, br	6 Connection: close
7 Content-Type: application/x-www-form-urlencoded	7
8 Content-Length: 53	8 <h2>
9 Origin: http://127.0.0.1:5000	Login successful
10 Connection: keep-alive	</h2>
11 Referer: http://127.0.0.1:5000/	<p>
12 Cookie: language=en; welcomebanner_status=dismiss	Welcome admin
13 Upgrade-Insecure-Requests: 1	</p>
14 Sec-Fetch-Dest: document	
15 Sec-Fetch-Mode: navigate	
16 Sec-Fetch-Site: same-origin	
17 Sec-Fetch-User: ?1	
18 Priority: u=0, i	
19	
20 username=admin' OR '1'='1' -- &password=wrongpassword	

Figure 1: SQL Injection: modified login request and successful authentication response

The captured response demonstrated that the authentication logic could be influenced by SQL syntax supplied through the username field. The successful response despite the incorrect password provided direct evidence that the intended authentication condition had been bypassed.

The security impact of this weakness is significant because authentication is a trust boundary. If the same flaw existed in a production application, an attacker could potentially bypass authentication or manipulate database query behavior depending on the query structure and database permissions.

Remediation:

The vulnerable query construction was replaced with parameterized database access so that supplied credentials are treated as data rather than executable SQL syntax. User input should also be constrained according to the expected format, and the application database account should operate with the minimum permissions required.

Retest:

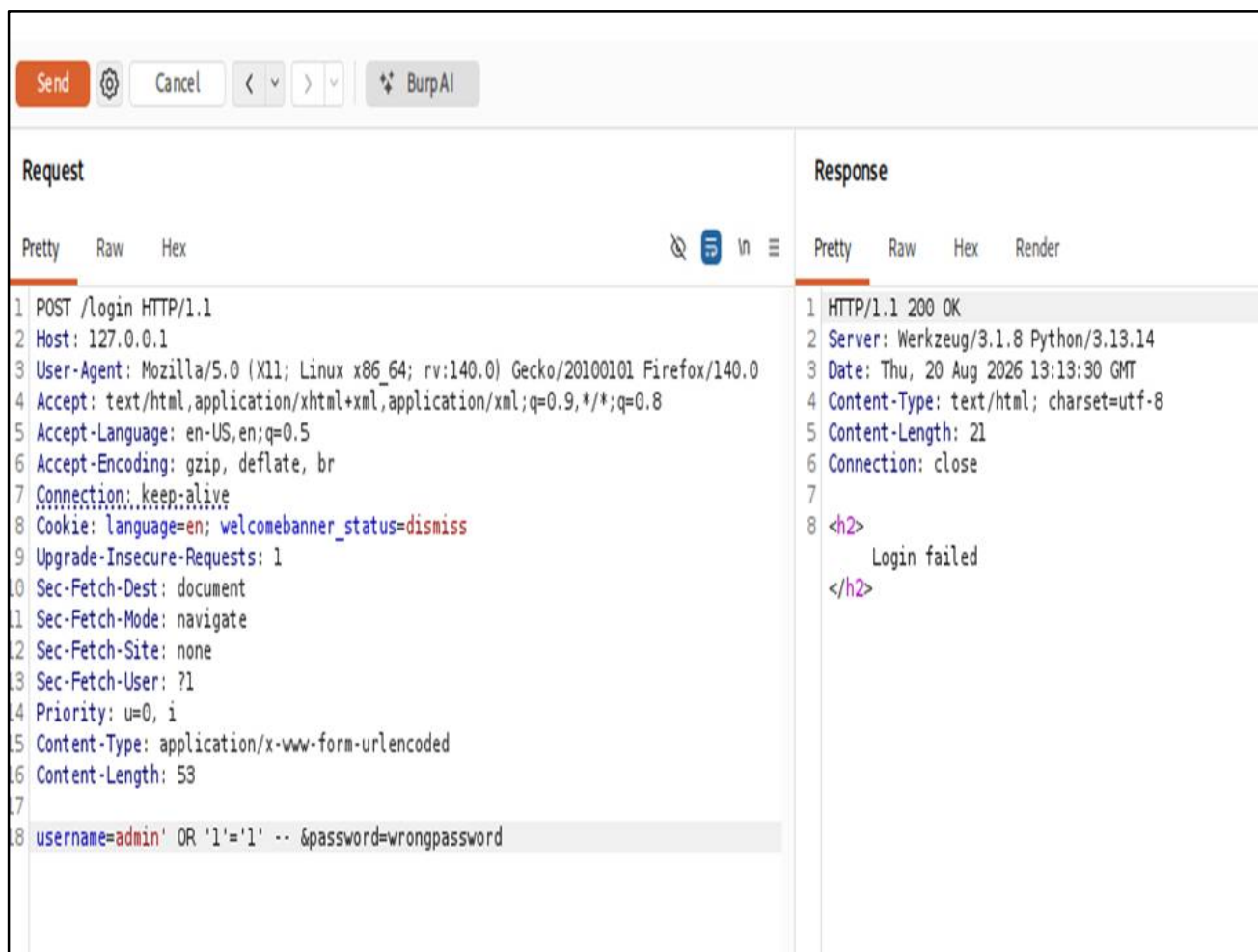


Figure 2: SQL Injection retest / comparison showing legitimate authentication behavior

After remediation, the injection attempt no longer produced the previously observed authentication bypass. A legitimate account test using the correct credentials continued to authenticate successfully, demonstrating that the fix preserved intended functionality while removing the demonstrated bypass condition.

8.2. Cross-Site Scripting

The search functionality accepted a user-controlled query parameter and returned the value in the application response. A script payload was supplied through the search parameter and the browser executed the JavaScript, producing an alert containing the text XSS. This established that the application was allowing untrusted input to reach a browser execution context.

Test input:

```
<script>alert('XSS')</script>
```

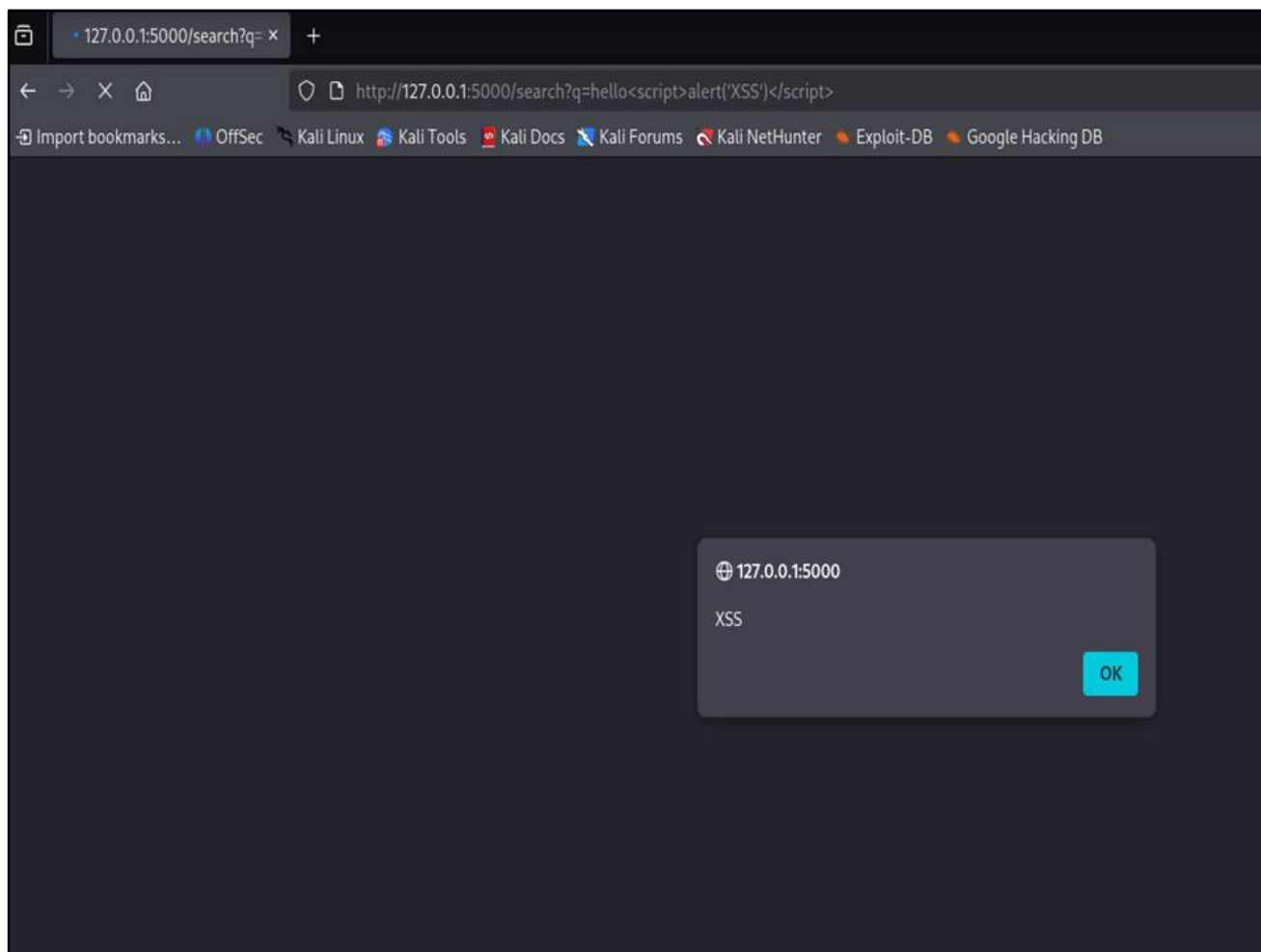


Figure 3: Cross-Site Scripting: browser alert demonstrating JavaScript execution

The observed alert was the key evidence for the vulnerability because the browser interpreted the supplied script rather than displaying it only as harmless text. In this lab, the test demonstrates client-side script execution caused by unsafe handling of user-controlled content.

The impact of a successful XSS condition depends on the application context in which the injected content is rendered. In a real application, script execution could potentially perform actions in the victim's browser context, access data available to page scripts, or modify the displayed content.

Remediation:

The search output was changed so that user-controlled content is rendered as text rather than being interpreted as executable HTML or JavaScript. Context-appropriate output encoding and safe templating should be used consistently, while a suitable Content Security Policy can provide an additional defensive layer.

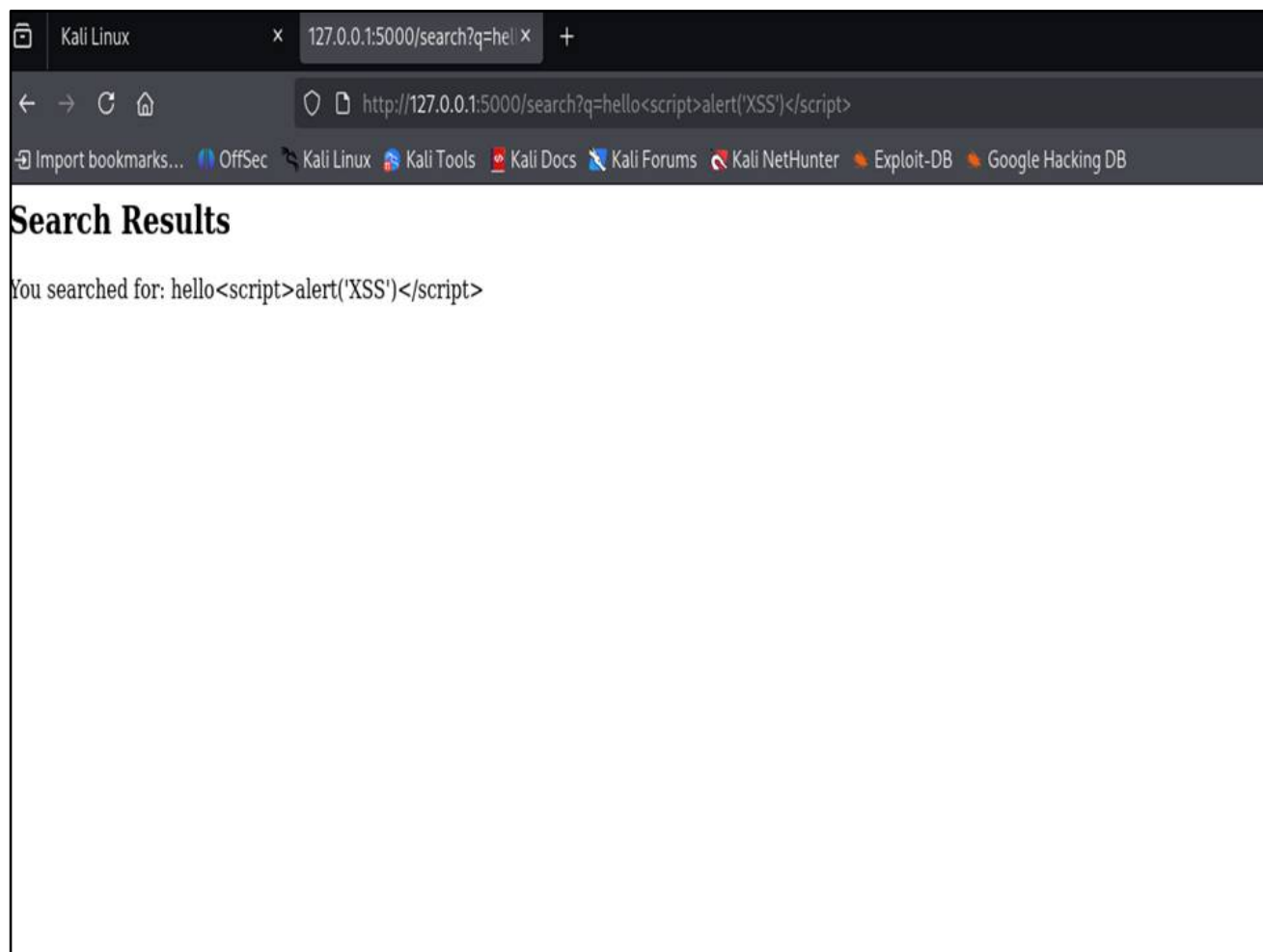
Retest:

Figure 4: XSS retest showing the payload rendered as text rather than executed

The post-remediation test confirmed that the same script input was no longer executed by the browser. This provided evidence that the vulnerable rendering behavior had been removed.

8.3. Broken Access Control

The profile endpoint initially accepted a user identifier through the URL and returned the corresponding user record. In the vulnerable implementation, changing the identifier was sufficient to request another user's profile without a server-side authorization decision tied to the authenticated session.

Initial profile request:

GET /profile?id=1

Host	Method	URL	Params	Status code ^	Length	MIME type
http://127.0.0.1:5000	GET	/		200	769	HTML
http://127.0.0.1:5000	POST	/login	✓	200	195	XML
http://127.0.0.1:5000	POST	/login	✓	200	195	XML
http://127.0.0.1:5000	GET	/profile?id=1	✓	200	266	text
http://127.0.0.1:5000	GET	/search?q=%3Cscript%3Ealert(%2...	✓	200	275	HTML
http://127.0.0.1:5000	GET	/search?q=hello	✓	200	250	text
http://127.0.0.1:5000	GET	/search?q=hello%3Cscript%3Ealer...	✓	200	280	HTML
http://127.0.0.1:5000	GET	/login				
http://127.0.0.1:5000	POST	/login	✓			
http://127.0.0.1:5000	GET	/profile				
http://127.0.0.1:5000	GET	/profile?id=2	✓			

Request		Response	
Pretty	Raw	Pretty	Raw
<pre> 1 GET /profile?id=1 HTTP/1.1 2 Host: 127.0.0.1:5000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Connection: keep-alive 8 Cookie: language=en; welcomebanner_status=dismiss 9 Upgrade-Insecure-Requests: 1 10 Sec-Fetch-Dest: document 11 Sec-Fetch-Mode: navigate 12 Sec-Fetch-Site: none 13 Sec-Fetch-User: ?1 14 Priority: u=0, i 15 16 </pre>		<pre> 1 HTTP/1.1 200 OK 2 Server: Werkzeug/3.1.8 Python/3.13.14 3 Date: Thu, 20 Aug 2026 11:58:44 GMT 4 Content-Type: text/html; charset=utf-8 5 Content-Length: 92 6 Connection: close 7 8 9 <h2> User Profile </h2> 10 <p> User ID: 1 </p> 11 <p> Username: admin </p> 12 </pre>	

Figure 5: Broken Access Control: request for profile id=1 returning another user's profile

A second request for the user's own profile demonstrated the expected mapping between the test account and its profile.

Comparison request:

GET /profile?id=2

Host	Method	URL	Params	Status code ^	Length	MIME type	Title
http://127.0.0.1:5000	GET	/		200	769	HTML	
http://127.0.0.1:5000	POST	/login	✓	200	195	XML	
http://127.0.0.1:5000	POST	/login	✓	200	195	XML	
http://127.0.0.1:5000	GET	/profile?id=1	✓	200	266	text	
http://127.0.0.1:5000	GET	/profile?id=2	✓	200	267	text	
http://127.0.0.1:5000	GET	/search?q=%3Cscript%3Ealert(%2...	✓	200	275	HTML	
http://127.0.0.1:5000	GET	/search?q=hello	✓	200	250	text	
http://127.0.0.1:5000	GET	/search?q=hello%3Cscript%3Ealer...	✓	200	280	HTML	
http://127.0.0.1:5000	GET	/login					
http://127.0.0.1:5000	POST	/login	✓				
http://127.0.0.1:5000	GET	/profile					

Request	Response
Pretty Raw Hex 1 GET /profile?id=2 HTTP/1.1 2 Host: 127.0.0.1:5000 3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8 5 Accept-Language: en-US,en;q=0.5 6 Accept-Encoding: gzip, deflate, br 7 Connection: keep-alive 8 Cookie: language=en; welcomebanner_status=dismiss 9 Upgrade-Insecure-Requests: 1 10 Sec-Fetch-Dest: document 11 Sec-Fetch-Mode: navigate 12 Sec-Fetch-Site: none 13 Sec-Fetch-User: ?1 14 Priority: u=0, i 15 16	Pretty Raw Hex Render 1 HTTP/1.1 200 OK 2 Server: Werkzeug/3.1.8 Python/3.13.14 3 Date: Thu, 20 Aug 2026 12:00:36 GMT 4 Content-Type: text/html; charset=utf-8 5 Content-Length: 93 6 Connection: close 7 8 9 User Profile 10 User ID: 2 11 Username: pritam 12

Figure 6: Profile comparison showing User ID 2 and username pritam

The important security issue was not that profile identifiers existed, but that the server trusted the identifier without verifying whether the authenticated user was authorized to access the referenced object. This is an object-level authorization problem.

Remediation:

The profile endpoint was changed so that authorization is determined on the server. The application now associates the request with the authenticated user and does not allow the URL parameter to override that identity. A request attempting to access a different user's profile is rejected rather than returning the requested user's record.

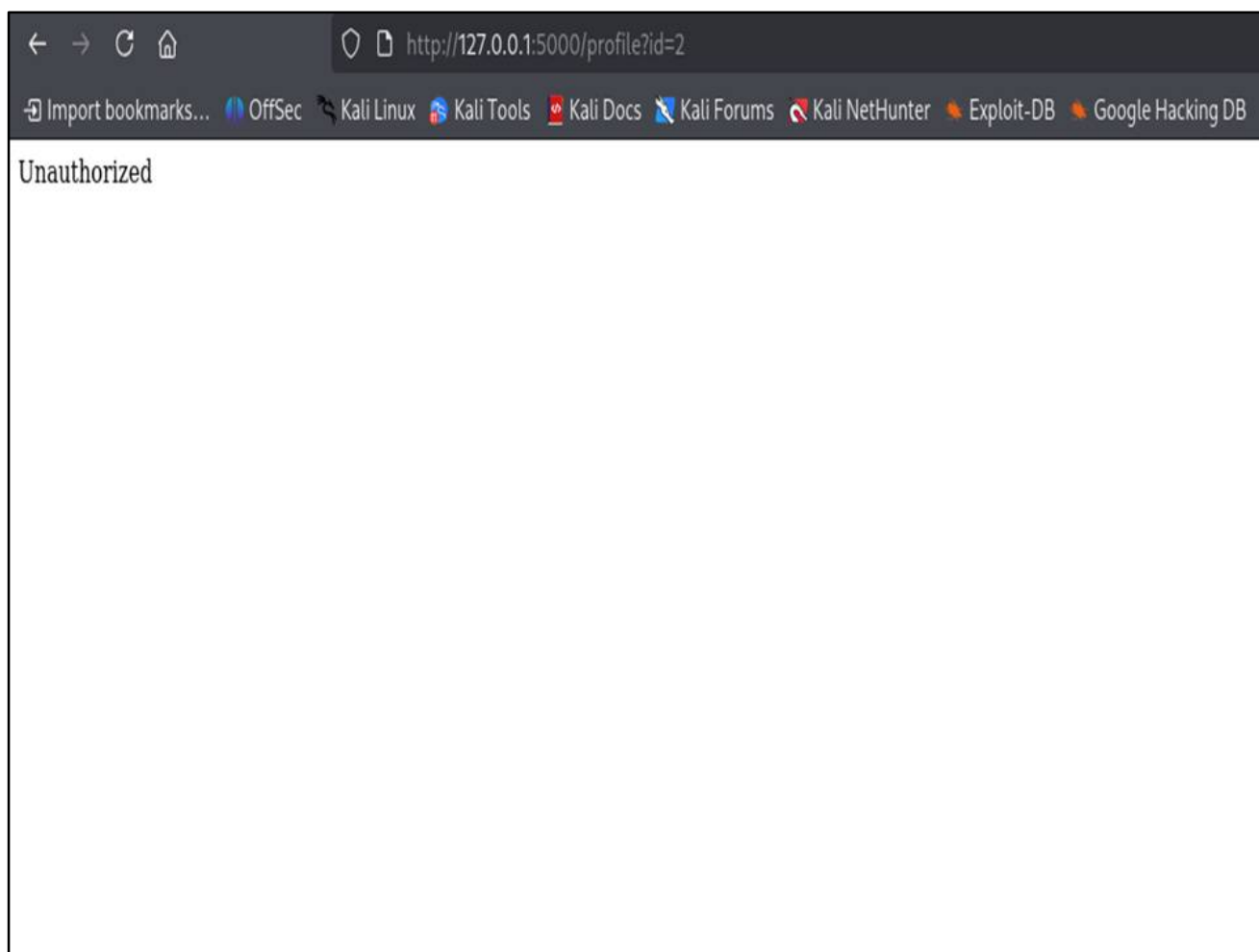


Figure 7: Broken Access Control retest showing an unauthorized response for another user's profile

A legitimate login was also verified using the test account so that the authorization fix could be evaluated under an authenticated session rather than simply blocking the entire profile function.

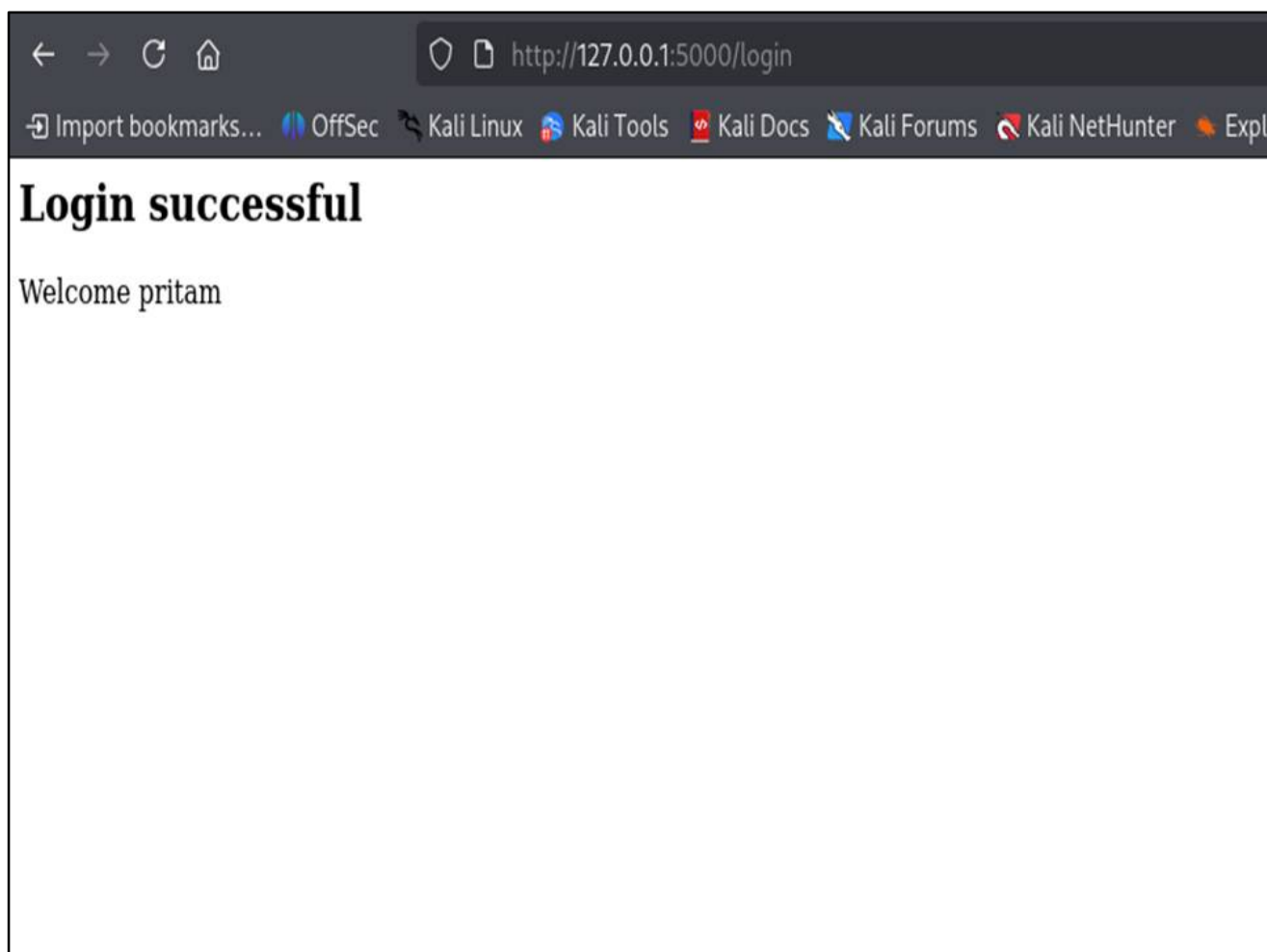


Figure 8: Legitimate authentication after remediation using the Pritam test account

The final profile request confirmed that the authenticated user's own identity was returned even when a different identifier was supplied in the URL.

Final authorization verification:

`http://127.0.0.1:5000/profile?id=1`

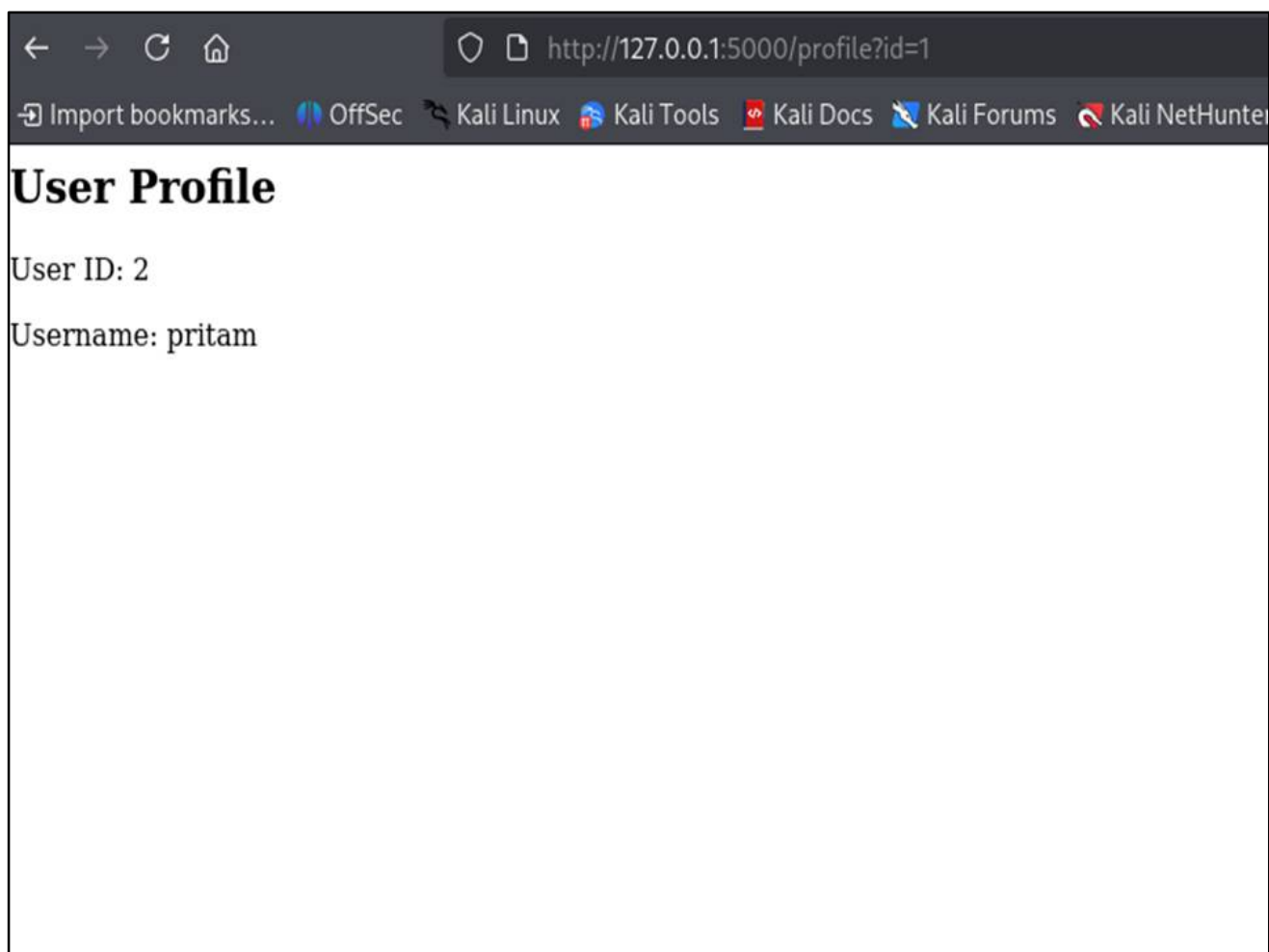


Figure 9: Final authorization retest showing User ID 2 / Username pritam despite id=1

This final result demonstrates the intended access-control behavior: the client-controlled object identifier no longer determines which user's profile is returned. The server-side identity and authorization decision take precedence.

9. Risk Summary

The following ratings are project-level assessments based on the behavior demonstrated in the controlled laboratory. They are not formal CVSS scores.

Finding	Risk	Reason
SQL Injection	High	Authentication query behavior could be influenced by user-controlled SQL syntax.
Cross-Site Scripting	Medium	User-controlled input was executed in the browser before remediation.
Broken Access Control	High	A client-controlled identifier initially allowed access to another user's profile.

10. Remediation and Retesting

The remediation phase focused on correcting the root causes rather than simply changing the test payloads. The SQL injection weakness was addressed through parameterized database queries. The XSS weakness was addressed through safe rendering of untrusted search content. The broken access-control weakness was addressed by making the server determine the authenticated user's identity and enforcing object-level authorization.

Retesting was performed against the same application functions after the changes. The SQL injection test no longer produced the demonstrated authentication bypass, the XSS payload no longer executed as JavaScript, and the profile endpoint rejected an attempt to use a different object identifier to obtain another user's profile. Legitimate authentication and profile access remained functional.

11. Conclusion

This assessment provided a practical end-to-end demonstration of web application security testing in a controlled local environment. The work began with application mapping and HTTP traffic inspection and progressed into manual testing of the application's authentication mechanism, user-controlled search functionality, and profile access controls. Using Burp Suite and the locally hosted Flask application made it possible to observe the requests generated by the browser, modify relevant parameters, and verify the resulting server behavior. Three important web application security weaknesses were demonstrated during the assessment. The SQL injection test showed that specially crafted input could influence the login query and result in authentication bypass behavior. The cross-site scripting test demonstrated that user-controlled search input could reach a browser execution context and cause JavaScript to execute. The broken access control test showed that a client-controlled profile identifier could initially be used to request another user's profile without an appropriate server-side authorization decision. These findings represented different security boundaries involving database query handling, browser-side input handling, and server-side authorization. The project did not stop at demonstrating the vulnerabilities. Each finding was analyzed in terms of its underlying cause and the appropriate remediation approach. Parameterized database queries were used to prevent user input from being interpreted as SQL syntax. The search functionality was corrected so that untrusted content was handled as data rather than executable browser content. The profile functionality was also changed so that authorization was determined by the authenticated user's identity on the server instead of trusting a user-controlled object identifier. Retesting was then performed against the corrected application to verify the effectiveness of the changes. The SQL injection attempt no longer produced the previously demonstrated authentication bypass, while legitimate authentication remained functional. The XSS payload was no longer executed as JavaScript after the rendering behavior was corrected. For broken access control, an attempt to request a different profile was rejected, while the authenticated user's own profile information remained accessible. These retests provided evidence that the remediation addressed the observed behavior rather than simply documenting a theoretical fix. Overall, the project demonstrated the complete security assessment cycle of identifying an attack surface, observing normal application behavior, testing security controls, collecting evidence, understanding vulnerability impact, applying remediation, and performing validation after the changes. It also provided practical experience with HTTP request manipulation, authentication testing, input handling, object-level authorization, and security-focused retesting. The assessment therefore demonstrates not only how vulnerabilities can be identified in a deliberately vulnerable application, but also how a security tester can communicate their cause, impact, remediation, and verification in a structured technical report.

12. References

OWASP Foundation, OWASP Top 10: Web Application Security Risks.

OWASP Foundation, OWASP Web Security Testing Guide (WSTG).

PortSwigger, Burp Suite Documentation and Web Security Academy.

Flask Documentation, Application and Request Handling Documentation.

Python Documentation, sqlite3 Database Interface.

Project laboratory source code and locally captured HTTP evidence.

Prepared by Pritam Bose